

## 0.1 Thu thập và xử lý dữ liệu

### 0.1.1 Dữ liệu phát hiện biển số

Tập dữ liệu dùng cho bài toán phát hiện biển số được tổ chức theo định dạng YOLOv8 OBB. Khác với bounding box ngang thông thường, mỗi nhãn trong tập dữ liệu gồm một lớp và bốn điểm của biển số:

```
<class_id> x1 y1 x2 y2 x3 y3 x4 y4
```

Các tọa độ được chuẩn hóa về khoảng  $[0, 1]$ . Cách biểu diễn này phù hợp với bài toán phát hiện biển số trong video, vì biển số thường bị nghiêng do góc quay camera, hướng di chuyển của phương tiện hoặc phối cảnh đường phố. Việc dùng OBB giúp vùng biển số được bao sát hơn, giảm phần nền thừa khi crop và tạo đầu vào ổn định hơn cho bước OCR phía sau.

Tập dữ liệu được tổng hợp từ nhiều nguồn khác nhau nhằm tăng độ đa dạng về góc nhìn, điều kiện ánh sáng, loại phương tiện và bố cục biển số. Các nguồn chính gồm dữ liệu Kaggle, các frame trích xuất từ video/YouTube, ảnh phương tiện từ Platesmania, và tập UFPR-ALPR. Ngoài ra, tập dữ liệu được bổ sung các mẫu âm tính không chứa biển số để giảm hiện tượng detector phát hiện nhầm biển số ở vùng không chứa biển.

**Bảng 1:** Thống kê dữ liệu phát hiện biển số

| Nhóm dữ liệu    | Số ảnh | Ghi chú                                             |
|-----------------|--------|-----------------------------------------------------|
| Platesmania     | 26.836 | Nguồn lớn nhất, ảnh phương tiện thực tế             |
| Kaggle          | 4.578  | Ảnh biển số xe chụp tại các bãi đỗ xe               |
| Các nguồn video | 2.851  | Frame trích xuất từ video                           |
| UFPR-ALPR       | 4.500  | Dùng để tăng đa dạng hình học cho bài toán định vị  |
| Mẫu âm tính     | 1.700  | Ảnh không chứa biển số, dùng để giảm false positive |

Tổng cộng, tập dữ liệu gồm 40.465 ảnh và 48.431 nhãn OBB. Dữ liệu được chia thành 33.333 ảnh huấn luyện và 7.132 ảnh kiểm định, tương ứng khoảng 82,4% / 17,6%. Trong toàn bộ tập dữ liệu có 38.261 ảnh có ít nhất một biển số và 2.204 ảnh âm tính với file nhãn rỗng. Các ảnh âm tính này vẫn được giữ lại vì trong hệ thống thực tế, detector thường nhận đầu vào là crop phương tiện hoặc frame video có thể không chứa biển số rõ ràng; nếu chỉ huấn luyện trên ảnh dương tính, mô hình dễ sinh dự đoán sai trên các vùng như đèn xe, logo, mặt ca-lăng hoặc biển quảng cáo.

### 0.1.2 Dữ liệu nhận diện biển số

Tập dữ liệu dùng cho bài toán OCR biển số gồm hai phần train và valid. Mỗi ảnh là một crop biển số đã được tách khỏi ảnh phương tiện hoặc frame gốc. Nhãn OCR được mã hóa trực tiếp trong tên file: phần trước dấu # là chuỗi biển số, phần

sau dấu # là định danh mẫu. Các biển số phần lớn được crop từ ảnh phương tiện thu thập trên Platesmania; phần còn lại đến từ Kaggle.

Ví dụ về tên nhãn dữ liệu:

79A[SEP]122.35#nomer12660750.jpg

30A-827.34#6748.jpg

KP-51-53#6542.jpg

Trong đó, [SEP] biểu diễn điểm ngắt dòng đối với biển số hai dòng. Cách mã hóa này phù hợp với mô hình OCR end-to-end, vì mô hình học trực tiếp ánh xạ từ ảnh crop sang chuỗi ký tự, không cần annotation bounding box cho từng ký tự.

Thống kê tổng quan của tập dữ liệu như sau:

**Bảng 2:** Thống kê tập dữ liệu OCR

| Thuộc tính                | Số lượng |
|---------------------------|----------|
| Tổng số lượng             | 33.376   |
| Train                     | 27.841   |
| Valid                     | 5.535    |
| Ảnh từ Platesmania        | 18.925   |
| Ảnh từ Kaggle             | 14.451   |
| Số chuỗi biển số duy nhất | 25.438   |

Tập dữ liệu có cả biển một dòng và biển hai dòng. Trong đó, biển hai dòng chiếm tỷ lệ lớn hơn:

**Bảng 3:** Thống kê bố cục

| Kiểu bố cục | Số ảnh | Tỉ lệ  |
|-------------|--------|--------|
| Một dòng    | 10.731 | 32,15% |
| Hai dòng    | 22.645 | 67,85% |

Tập ký tự được sử dụng để huấn luyện mô hình gồm 10 chữ số (0–9), 24 chữ cái Latin viết hoa xuất hiện trên biển số xe Việt Nam (A, B, C, D, E, F, G, H, K, L, M, N, P, Q, R, S, T, U, V, X, Y, Z, Đ), cùng các ký tự phân tách (-, ., [SEP]). Ngoài ra, tập ký tự còn bao gồm ký hiệu <blank> phục vụ thuật toán CTC.

### 0.1.3 Dữ liệu đánh giá chất lượng biển số

Bên cạnh dữ liệu phát hiện và nhận diện ký tự, hệ thống còn sử dụng một tập dữ liệu riêng cho bài toán đánh giá chất lượng ảnh crop biển số. Mục tiêu của mô hình này là ước lượng mức độ đọc được của biển số trước khi đưa ảnh sang OCR hoặc các bước xử lý tăng cường. Việc tách riêng bài toán chất lượng giúp pipeline

tránh xử lý mọi crop theo cùng một cách: ảnh rõ có thể được nhận diện trực tiếp, trong khi ảnh mờ, nhỏ hoặc khó đọc có thể được đưa vào nhánh chờ thêm khung hình hoặc nhánh hợp nhất đa khung.

Ở giai đoạn đầu, mô hình được huấn luyện trên dữ liệu LPLCv2 do nhóm tác giả cung cấp. LPLCv2 là tập dữ liệu phân loại độ đọc được của biển số với bốn mức nhãn `perfect`, `good`, `poor` và `illegible`, được xây dựng nhằm đánh giá không chỉ chất lượng ảnh nói chung mà trực tiếp là khả năng đọc ký tự trên biển số **wojcik2026lplcv2**. Từ dữ liệu gốc, luận văn tạo bản dùng huấn luyện theo bốn lớp chất lượng và chia thành hai tập `train/valid` để huấn luyện mô hình ban đầu.

**Bảng 4:** Thống kê dữ liệu đánh giá chất lượng biển số

| Tập dữ liệu      | Train  | Valid  | Tổng số ảnh |
|------------------|--------|--------|-------------|
| LPLCv2-quality   | 39.767 | 14.876 | 54.643      |
| Fine-tune nội bộ | 24.796 | 5.993  | 30.789      |

Sau khi có mô hình khởi tạo từ LPLCv2, luận văn fine-tune thêm trên tập dữ liệu nội bộ được gom từ các crop biển số gần với miền dữ liệu của hệ thống hơn. Các ảnh này vẫn được tổ chức theo cùng bốn mức chất lượng, nhờ đó mô hình giữ được định nghĩa nhãn nhất quán nhưng thích nghi tốt hơn với ảnh biển số Việt Nam, điều kiện camera thực tế và các lỗi thường gặp trong pipeline như crop lệch, nhòe chuyển động hoặc biển số quá nhỏ.

**Bảng 5:** Phân bố nhãn trong dữ liệu chất lượng

| Tập dữ liệu      | Perfect | Good   | Poor   | Illegible |
|------------------|---------|--------|--------|-----------|
| LPLCv2-quality   | 24.271  | 13.458 | 9.898  | 7.016     |
| Fine-tune nội bộ | 7.117   | 12.198 | 10.130 | 1.344     |

## 0.2 Huấn luyện và fine-tuning mô hình

### 0.2.1 Môi trường huấn luyện

Các mô hình được huấn luyện trên máy trạm trang bị GPU NVIDIA GeForce RTX 4050. Phần mềm huấn luyện dựa trên nền tảng Python 3.10, PyTorch 2.6.0 và CUDA 12.4.

### 0.2.2 Mô hình phát hiện biển số YOLOv8-OB

Mô hình phát hiện biển số YOLOv8-OB được huấn luyện theo tác vụ phát hiện hộp bao có hướng, tương ứng với cấu hình `task=obj` trong Ultralytics. Mô hình được khởi tạo với trọng số ngẫu nhiên theo cơ chế mặc định của Ultralytics/PyTorch, các thiết lập chính của quá trình huấn luyện được trình bày trong Bảng ??.

**Bảng 6:** Cấu hình huấn luyện mô hình phát hiện biển số dạng OBB

| Nhóm cấu hình            | Giá trị                                                                                    |
|--------------------------|--------------------------------------------------------------------------------------------|
| Tác vụ                   | task=obb                                                                                   |
| Checkpoint khởi tạo      | Khởi tạo ngẫu nhiên                                                                        |
| Kích thước ảnh đầu vào   | 640 × 640                                                                                  |
| Số epoch                 | 50                                                                                         |
| Batch size               | 8                                                                                          |
| Thiết bị huấn luyện      | GPU, device=0                                                                              |
| Số workers               | 8                                                                                          |
| AMP                      | True                                                                                       |
| Thiết lập tái lập        | seed=0, deterministic=True                                                                 |
| Bộ tối ưu                | auto                                                                                       |
| Learning rate            | lr0=0.001, lrf=0.01                                                                        |
| Momentum và weight decay | momentum=0.937,<br>weight_decay=0.0005                                                     |
| Warm-up                  | warmup_epochs=3,<br>warmup_momentum=0.8,<br>warmup_bias_lr=0.1                             |
| Trọng số loss            | box=7.5, cls=0.5, dfl=1.5, angle=1.0                                                       |
| Tăng cường dữ liệu       | hsv_h=0.015, hsv_s=0.7,<br>hsv_v=0.4, translate=0.1,<br>scale=0.5, flip_lr=0.5, mosaic=1.0 |
| Các phép không sử dụng   | mixup, cutmix, copy_paste,<br>flipud, shear, perspective                                   |
| Close mosaic             | close_mosaic=10                                                                            |

### 0.2.3 Huấn luyện và fine-tune Quality Router

Mô hình Quality Router được xây dựng dựa trên kiến trúc phân loại ảnh nhẹ YOLOv8n-cls. Quá trình huấn luyện sử dụng tập dữ liệu đánh giá chất lượng được trình bày ở Mục ?? và diễn ra theo hai giai đoạn: đầu tiên mô hình được huấn luyện trên dữ liệu LPLCv2 để học khái niệm độ đọc được (legibility) tổng quát, sau đó được fine-tune trên tập ảnh crop biển số thực tế tại Việt Nam nhằm thích nghi với miền dữ liệu của hệ thống. Mô hình duy trì cấu trúc dự đoán bốn nhãn phân loại: *perfect*, *good*, *poor* và *illegible*.

Bên cạnh bài toán bốn lớp, một biến thể nhị phân cũng được huấn luyện như một thí nghiệm phụ, tuy nhiên hệ thống chốt sử dụng mô hình bốn lớp để phục vụ cơ chế ánh xạ ba nhánh (*direct*, *tracklet\_fusion*, *unreadable\_wait*) trong pipeline. Cấu hình huấn luyện chi tiết cho giai đoạn fine-tune được trình bày trong Bảng ??.

**Bảng 7:** Cấu hình huấn luyện mô hình Quality Router (YOLOv8n-cls)

| Nhóm cấu hình            | Giá trị                                                                        |
|--------------------------|--------------------------------------------------------------------------------|
| Tác vụ                   | task=classify                                                                  |
| Kiến trúc backbone       | YOLOv8n-cls                                                                    |
| Kích thước ảnh đầu vào   | 64 × 64                                                                        |
| Số epoch                 | 50 (Early stopping với patience=10)                                            |
| Batch size               | 16                                                                             |
| Bộ tối ưu                | auto (lr0=0.0001, lrf=0.01)                                                    |
| Momentum và weight decay | momentum=0.937,<br>weight_decay=0.0005                                         |
| Warm-up                  | warmup_epochs=3.0,<br>warmup_momentum=0.8,<br>warmup_bias_lr=0.0               |
| Tăng cường dữ liệu       | hsv_h=0.015, hsv_s=0.7,<br>hsv_v=0.4, translate=0.1,<br>scale=0.5, flip_lr=0.5 |
| Auto-augment & Erasing   | auto_augment=randaugment, erasing=0.4                                          |
| Thiết bị & Môi trường    | GPU, device=0, 8 workers, AMP True                                             |
| Tính tái lập             | seed=0, deterministic=True                                                     |

Kích thước đầu vào  $64 \times 64$  được lựa chọn để tối ưu tốc độ suy luận, phù hợp với yêu cầu xử lý video thời gian thực. Các kỹ thuật tăng cường dữ liệu đặc thù như Random Erasing (erasing=0.4) và RandAugment được áp dụng nhằm mô phỏng các biến dạng lóa sáng, nhiễu hạt và che khuất một phần thường gặp trên ảnh crop biển số, giúp mô hình tăng cường khả năng khái quát hóa. Chi tiết về độ chính xác (Accuracy) và ma trận nhầm lẫn của mô hình được đánh giá trong Mục ??.

## 0.2.4 Huấn luyện SmallLPR-Line-CTC

**DataLoader và pipeline nạp dữ liệu.** Dữ liệu được nạp qua SmallLPRLLineCTC-DataModule của PyTorch Lightning, sử dụng DataLoader với batch\_size=64, num\_workers=4 và pin\_memory=True. Hàm collate\_fn\_line\_ctc đóng gói mỗi batch thành một dictionary duy nhất chứa: ảnh đầu vào dạng tensor (B, 3, 48, 96); các target CTC đã được flatten và padding theo từng nhánh (global\_targets, one\_line\_targets, top\_targets, bottom\_targets); độ dài target tương ứng (global\_lengths, one\_line\_lengths, ...); input lengths cố định cho mỗi nhánh (72 bước cho global, 16 cho one-line, 12 cho top/bottom); mask cho từng nhánh (one\_line\_loss\_mask, top\_loss\_mask, bottom\_loss\_mask) xác định mẫu nào đóng góp vào loss tương ứng; và nhãn layout (layout\_labels) kèm mask layout\_loss\_mask để bỏ qua các mẫu không xác định bố cục. Tập huấn luyện được shuffle, tập kiểm định giữ nguyên thứ

tự.

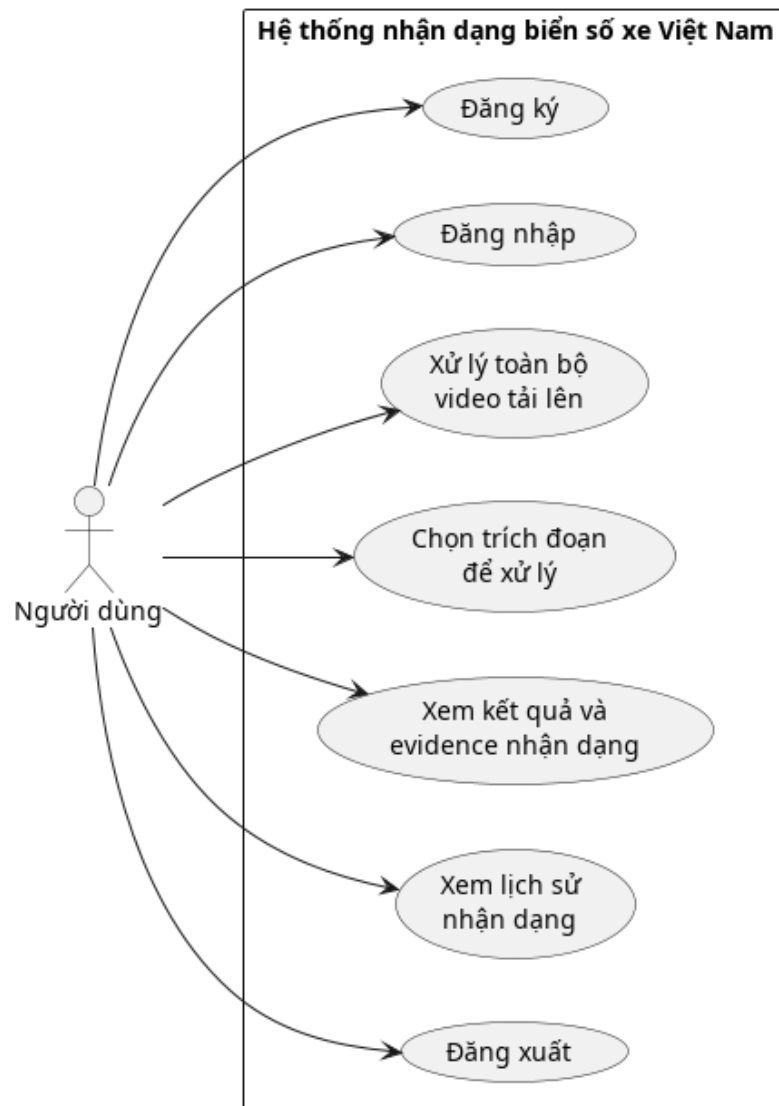
**Thiết lập huấn luyện.** Quá trình huấn luyện được điều phối bởi hàm `train()` trong `scripts/train_small_lpr_line_ctc.py`, sử dụng PyTorch Lightning Trainer với các tham số chính: `accelerator="auto"` (tự chọn CUDA nếu có), `devices=1`, `precision=32`, `max_epochs=200`, `gradient_clip_val=1.0`. Seed toàn cục được cố định (`seed=42`) qua `L.seed_everything(42, workers=True)` để đảm bảo tính tái lập. Cấu hình được quản lý tập trung qua file YAML `config/small_lpr_line_ctc_config.yaml`, có thể ghi đè từ dòng lệnh thông qua các tham số như `-epochs`, `-batch-size`, `-lr`, `-use-stn`, `-use-pos-enc`, `-line-prior-strength`, `-decode-mode`.

**Callbacks và theo dõi huấn luyện.** Ba callback chính được sử dụng: `ModelCheckpoint` lưu top-3 checkpoint có `val_acc` cao nhất và checkpoint cuối cùng (`save_last=True`) vào thư mục có timestamp, tên file theo mẫu `small_lpr_line_ctc-{epoch}-{val_acc}.ckpt`; `EarlyStopping` dừng huấn luyện nếu `val_acc` không cải thiện trong 40 epoch liên tiếp; `LearningRateMonitor` ghi lại giá trị learning rate mỗi epoch. Logging được thực hiện qua `CSVLogger` lưu vào thư mục checkpoint, ghi lại tất cả metric huấn luyện và kiểm định: loss từng nhánh CTC, layout loss, layout accuracy, sequence accuracy.

**Chiến lược tái lập và khởi tạo.** Hàm `train()` hỗ trợ hai chế độ khởi tạo trọng số: `-resume` tiếp tục huấn luyện từ checkpoint đã có, giữ nguyên trạng thái optimizer và scheduler; `-init-from` khởi tạo model từ checkpoint của mô hình khác (ví dụ: nạp trọng số từ `small_lpr` cũ) rồi bắt đầu một vòng huấn luyện mới với optimizer mới. Cơ chế `_with_legacy_one_line_weights()` xử lý tương thích ngược: nếu checkpoint nguồn không có `one_line_head`, hệ thống tự sao chép trọng số từ `global_head` sang `one_line_head` và khởi tạo `one_line_attention` mới. Hai chế độ này không được dùng đồng thời — việc sử dụng cả hai sẽ sinh lỗi.

### 0.3 Yêu cầu và kiến trúc tổng quan

#### 0.3.1 Sơ đồ use case



**Hình 0.1:** Sơ đồ use case tổng quan

**Tác nhân tham gia** Hệ thống có một tác nhân duy nhất là Người dùng đại diện cho bất kỳ cá nhân nào truy cập hệ thống qua giao diện web. Tác nhân này đóng hai vai trò tùy theo trạng thái xác thực: với các chức năng quản lý tài khoản, người dùng là khách chưa được xác thực; với các chức năng nghiệp vụ chính như xử lý video, phân tích sự kiện từ video, người dùng phải đã đăng nhập hợp lệ. Hệ thống không phân biệt vai trò theo nhóm quyền mà kiểm soát truy cập hoàn toàn dựa trên phiên đăng nhập.

**Các use case chính** Hệ thống có bảy use case chia thành hai nhóm chức năng.

Nhóm quản lý tài khoản gồm ba use case. *Đăng ký* cho phép người dùng tạo tài

khoản mới; hệ thống xác thực dữ liệu đầu vào, băm mật khẩu và lưu thông tin vào cơ sở dữ liệu. *Đăng nhập* cho phép người dùng xác thực danh tính và nhận token bảo mật để truy cập các chức năng được bảo vệ. *Đăng xuất* hủy phiên làm việc hiện tại và đưa người dùng về trạng thái khách.

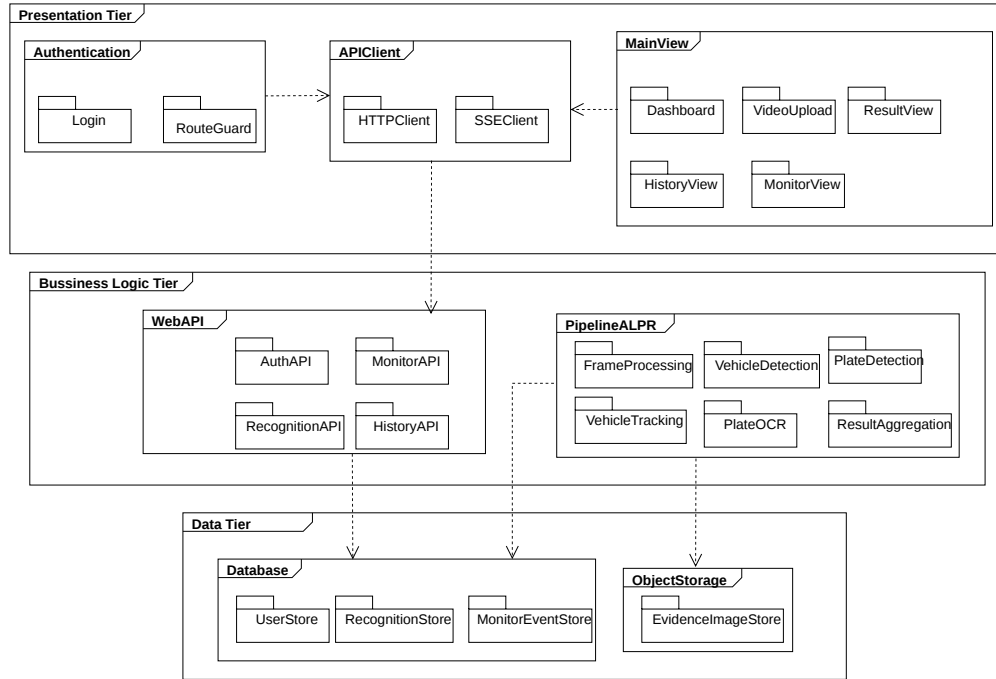
Nhóm nghiệp vụ nhận dạng gồm bốn use case, chỉ khả dụng sau khi đăng nhập. *Xử lý toàn bộ video tải lên* là luồng xử lý chính: người dùng tải lên một tệp video hoàn chỉnh, hệ thống chạy pipeline ALPR trên toàn bộ nội dung video và trả tiền độ, kết quả nhận dạng về giao diện theo thời gian thực. *Chọn trích đoạn để xử lý* phục vụ trường hợp người dùng chỉ cần phân tích một khoảng thời gian cụ thể từ nguồn camera trực tiếp hoặc từ video đã tải lên. Trong luồng này, người dùng chọn nguồn video, đánh dấu đoạn thời gian cần xử lý, sau đó backend trích xuất các khung hình tương ứng và chạy pipeline nhận dạng trên cửa sổ thời gian đã chọn. *Xem kết quả và minh chứng nhận dạng* cho phép người dùng xem lại kết quả của từng phiên xử lý, bao gồm danh sách biển số nhận dạng được, các xe bị từ chối kèm lý do và ảnh crop làm minh chứng. *Xem lịch sử nhận dạng* cung cấp danh sách các phiên xử lý đã thực hiện theo tài khoản đăng nhập, giúp người dùng tra cứu lại kết quả của các video hoặc trích đoạn đã xử lý trước đó.

### 0.3.2 Thiết kế kiến trúc

**Lựa chọn kiến trúc phần mềm** Hệ thống áp dụng kiến trúc **ba lớp** (Three-tier Architecture) kết hợp với mô hình Client–Server. Kiến trúc này phân tách ứng dụng thành ba tầng độc lập: tầng trình bày chịu trách nhiệm hiển thị giao diện và tương tác với người dùng, tầng xử lý nghiệp vụ thực thi toàn bộ logic ứng dụng, và tầng dữ liệu quản lý việc lưu trữ và truy vấn dữ liệu bền vững. Mỗi tầng chỉ giao tiếp với tầng liền kề thông qua giao diện được định nghĩa rõ ràng, giúp các tầng có thể thay đổi hoặc mở rộng độc lập mà không ảnh hưởng lẫn nhau.

Kiến trúc này được lựa chọn vì hệ thống có đặc thù xử lý nặng ở phía máy chủ (do sử dụng pipeline AI đa tầng với nhiều mô hình học sâu hoạt động nối tiếp), trong khi giao diện người dùng cần phản hồi nhanh và không bị chặn trong quá trình chờ kết quả. Việc tách biệt rõ tầng nghiệp vụ cũng cho phép cải tiến hay thay thế các mô hình AI mà không cần thay đổi giao diện hay cơ sở dữ liệu.





**Hình 0.2:** Biểu đồ phụ thuộc gói

## Biểu đồ phụ thuộc gói

**Tầng trình bày (Presentation Tier)** Tầng trình bày gồm ba package. Package *Authentication* chứa các màn hình xác thực người dùng, bao gồm màn hình đăng nhập và thành phần bảo vệ định tuyến, ngăn người dùng chưa xác thực truy cập các trang nghiệp vụ. Package *MainView* tổ chức toàn bộ giao diện sau đăng nhập, gồm trang tổng quan, màn hình tải video, màn hình xem kết quả và minh chứng, màn hình lịch sử và màn hình chọn trích đoạn từ nguồn video. Package *APIClient* đóng vai trò cầu nối duy nhất giữa tầng trình bày và tầng nghiệp vụ, cung cấp hai thành phần: client xử lý các yêu cầu REST đồng bộ và client nhận kết quả theo luồng thời gian thực từ server.

**Tầng nghiệp vụ (Business Logic Tier)** Tầng nghiệp vụ gồm hai package. Package *WebAPI* tiếp nhận và xử lý toàn bộ yêu cầu HTTP từ client, được tổ chức thành bốn router theo chức năng: *AuthAPI* quản lý đăng ký, đăng nhập và xác thực JWT; *RecognitionAPI* tiếp nhận video đầu vào, khởi động pipeline xử lý và phát kết quả về client qua SSE; *MonitorAPI* quản lý các phiên chọn trích đoạn từ camera trực tiếp hoặc video tải lên, đồng thời xử lý các sự kiện được người dùng đánh dấu;; *HistoryAPI* cung cấp dữ liệu lịch sử phiên nhận dạng theo tài khoản. Package *PipelineALPR* thực thi toàn bộ luồng xử lý AI, gồm năm sub-package hoạt động

theo chuỗi: FrameProcessing giải mã và tiền xử lý khung hình từ video đầu vào; VehicleDetection phát hiện phương tiện trong từng khung hình; VehicleTracking theo dõi từng phương tiện xuyên suốt các khung hình liên tiếp; PlateDetection phát hiện và trích xuất vùng biển số; PlateOCR nhận dạng ký tự biển số; ResultAggregation tổng hợp kết quả từ nhiều khung hình để đưa ra kết quả cuối cùng cho từng phương tiện.

**Tầng dữ liệu (Data Tier)** Tầng dữ liệu gồm hai package với trách nhiệm phân tách rõ ràng. Package Database lưu trữ toàn bộ dữ liệu có cấu trúc, gồm UserStore (thông tin tài khoản và phiên xác thực), RecognitionStore (kết quả các phiên nhận dạng và bản ghi từng phương tiện) và MonitorEventStore lưu thông tin các sự kiện do người dùng đánh dấu trên nguồn video, gồm nguồn dữ liệu, khoảng thời gian được chọn, trạng thái xử lý và kết quả nhận dạng. Package ObjectStorage lưu trữ dữ liệu nhị phân, cụ thể là EvidenceImageStore chứa ảnh crop biển số và ảnh minh chứng phương tiện; package Database chỉ lưu URL tham chiếu đến các tài nguyên này thay vì nhúng trực tiếp ảnh vào document.

## 0.4 Xây dựng ứng dụng

### 0.4.1 Công nghệ sử dụng

Hệ thống được phát triển bằng các công cụ, ngôn ngữ lập trình và thư viện trong Bảng ??.

**Bảng 8:** Danh sách thư viện và công cụ sử dụng

| Mục đích          | Công cụ/Thư viện   | Phiên bản | Địa chỉ URL                                                                 |
|-------------------|--------------------|-----------|-----------------------------------------------------------------------------|
| IDE lập trình     | Visual Studio Code | 1.104.0   | <a href="https://code.visualstudio.com/">https://code.visualstudio.com/</a> |
| Quản lý mã nguồn  | Git                | 2.43.0    | <a href="https://git-scm.com/">https://git-scm.com/</a>                     |
| Đóng gói ứng dụng | Docker             | 28.5.1    | <a href="https://www.docker.com/">https://www.docker.com/</a>               |
| Ngôn ngữ backend  | Python             | 3.10      | <a href="https://www.python.org/">https://www.python.org/</a>               |
| Runtime frontend  | Node.js            | 20        | <a href="https://nodejs.org/">https://nodejs.org/</a>                       |
| Package manager   | npm                | 11.6.3    | <a href="https://www.npmjs.com/">https://www.npmjs.com/</a>                 |

| <b>Mục đích</b>     | <b>Công cụ/Thư viện</b> | <b>Phiên bản</b> | <b>Địa chỉ URL</b>                                                                              |
|---------------------|-------------------------|------------------|-------------------------------------------------------------------------------------------------|
| Backend API         | FastAPI                 | $\geq 0.111.0$   | <a href="https://fastapi.tiangolo.com/">https://fastapi.tiangolo.com/</a>                       |
| ASGI server         | Uvicorn                 | $\geq 0.29.0$    | <a href="https://www.uvicorn.org/">https://www.uvicorn.org/</a>                                 |
| Học sâu             | PyTorch                 | 2.4.1            | <a href="https://pytorch.org/">https://pytorch.org/</a>                                         |
| Xử lý ảnh           | OpenCV Python           | 4.10.0.84        | <a href="https://opencv.org/">https://opencv.org/</a>                                           |
| Phát hiện đối tượng | Ultralytics YOLO        | $\geq 8.0.0$     | <a href="https://docs.ultralytics.com/">https://docs.ultralytics.com/</a>                       |
| Theo dõi đối tượng  | BoxMOT                  | 19.0.0           | <a href="https://github.com/mikel-brostrom/boxmot">https://github.com/mikel-brostrom/boxmot</a> |
| Suy luận ONNX       | ONNX Runtime            | 1.20.0           | <a href="https://onnxruntime.ai/">https://onnxruntime.ai/</a>                                   |
| Tính toán ma trận   | NumPy                   | 1.26.4           | <a href="https://numpy.org/">https://numpy.org/</a>                                             |
| Tính toán khoa học  | SciPy                   | $\geq 1.11.0$    | <a href="https://scipy.org/">https://scipy.org/</a>                                             |
| Huấn luyện mô hình  | Lightning               | 2.x              | <a href="https://lightning.ai/">https://lightning.ai/</a>                                       |
| Xử lý ảnh phụ trợ   | Pillow                  | $\geq 10.0.0$    | <a href="https://python-pillow.org/">https://python-pillow.org/</a>                             |
| Đọc cấu hình        | PyYAML                  | $\geq 6.0.1$     | <a href="https://pyyaml.org/">https://pyyaml.org/</a>                                           |
| Xác thực mật khẩu   | bcrypt                  | $\geq 4.1.0$     | <a href="https://github.com/pyca/bcrypt">https://github.com/pyca/bcrypt</a>                     |
| JWT                 | PyJWT                   | $\geq 2.8.0$     | <a href="https://pyjwt.readthedocs.io/">https://pyjwt.readthedocs.io/</a>                       |
| Cơ sở dữ liệu       | MongoDB                 | 7                | <a href="https://www.mongodb.com/">https://www.mongodb.com/</a>                                 |
| Driver mongoDB      | Motor                   | $\geq 3.3.0$     | <a href="https://motor.readthedocs.io/">https://motor.readthedocs.io/</a>                       |

| Mục đích            | Công cụ/Thư viện | Phiên bản    | Địa chỉ URL                                                                                 |
|---------------------|------------------|--------------|---------------------------------------------------------------------------------------------|
| Lưu trữ ảnh         | Supabase         | $\geq 2.5.0$ | <a href="https://supabase.com/">https://supabase.com/</a>                                   |
| Frontend framework  | React            | 18.3.1       | <a href="https://react.dev/">https://react.dev/</a>                                         |
| Điều hướng frontend | React Router DOM | 7.17.0       | <a href="https://reactrouter.com/">https://reactrouter.com/</a>                             |
| Build frontend      | Vite             | 5.4.21       | <a href="https://vite.dev/">https://vite.dev/</a>                                           |
| Thiết kế giao diện  | Tailwind CSS     | 3.4.19       | <a href="https://tailwindcss.com/">https://tailwindcss.com/</a>                             |
| Streaming video     | MediaMTX         | v1.19.1      | <a href="https://github.com/bluenviron/mediamtx">https://github.com/bluenviron/mediamtx</a> |
| Xử lý video         | FFmpeg           | 6.1.1        | <a href="https://ffmpeg.org/">https://ffmpeg.org/</a>                                       |
| Kiểm thử            | pytest           | –            | <a href="https://docs.pytest.org/">https://docs.pytest.org/</a>                             |

#### 0.4.2 Frontend

Frontend của hệ thống là ứng dụng web một trang được xây dựng bằng React và Vite. Giao diện không thực hiện suy luận mô hình trực tiếp mà đóng vai trò tiếp nhận đầu vào từ người dùng, gửi yêu cầu tới backend, nhận tiến độ xử lý qua cơ chế streaming và hiển thị kết quả nhận dạng theo từng phương tiện hoặc từng sự kiện.

Ứng dụng được tổ chức theo hai chế độ chính. Chế độ thứ nhất xử lý toàn bộ một video được tải lên, phù hợp với nhu cầu đánh giá kết quả nhận dạng trên một tệp video hoàn chỉnh. Chế độ thứ hai cho phép người dùng chọn một trích đoạn cần xử lý từ nguồn camera trực tiếp hoặc từ video đã tải lên. Người dùng chuyển đổi giữa hai chế độ này thông qua thành phần điều hướng trên thanh tiêu đề.

| Thành phần               | Vai trò                                                                                                                                                   |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quản lý xác thực         | Quản lý trạng thái đăng nhập, gọi các API xác thực và tự động gắn token bảo mật cho các yêu cầu ghi dữ liệu.                                              |
| Màn hình làm việc chính  | Điều phối trạng thái chính: file video, tiến độ xử lý, danh sách phương tiện hợp lệ và danh sách kết quả bị loại.                                         |
| Bảng điều khiển nguồn    | Cho phép người dùng chọn file video và các tùy chọn xử lý trước khi gửi yêu cầu lên server.                                                               |
| Khung hiển thị video     | Hiển thị video gốc hoặc khung hình đã chú thích được server gửi về trong quá trình xử lý.                                                                 |
| Bảng kết quả nhận dạng   | Hiển thị kết quả theo từng phương tiện, gồm biểu số tổng hợp, độ tin cậy, ảnh minh chứng và các khung hình trong bộ đệm theo dõi.                         |
| Màn hình lịch sử         | Truy vấn và hiển thị danh sách các phiên xử lý đã lưu và kết quả chi tiết của từng phiên.                                                                 |
| Màn hình chọn trích đoạn | Quản lý các phiên chọn trích đoạn từ nguồn camera trực tiếp hoặc video tải lên, tiếp nhận lệnh đánh dấu sự kiện và hiển thị kết quả nhận dạng theo luồng. |

**Bảng 9:** Các thành phần giao diện chính

Trong chế độ xử lý toàn bộ video, người dùng chọn file tại bảng điều khiển nguồn. Sau khi gửi yêu cầu, hệ thống nhận về một mã định danh công việc và mở kết nối streaming để nhận các sự kiện: tiến độ theo frame, khung hình đang xử lý, kết quả từng phương tiện và trạng thái hoàn thành. Dữ liệu nhận được được cập nhật vào trạng thái của màn hình chính, nhờ vậy giao diện hiển thị tiến độ và kết quả mà không cần tải lại trang.

Trong chế độ chọn trích đoạn, người dùng trước hết chọn nguồn dữ liệu, có thể là camera trực tiếp hoặc video tải lên. Khi người dùng đánh dấu một khoảng thời gian cần xử lý, frontend gửi thông tin phiên, loại nguồn và mốc thời gian đã chọn tới server. Backend sau đó trích xuất các khung hình thuộc khoảng thời gian này, chạy pipeline nhận dạng biểu số và đẩy tiến độ cùng kết quả về frontend theo cơ chế streaming. Cách tổ chức này giúp thao tác chọn đoạn trên giao diện diễn ra nhanh, trong khi phần xử lý AI được thực hiện bất đồng bộ ở phía server.

### 0.4.3 Backend

Backend được xây dựng bằng FastAPI với một điểm khởi động duy nhất chịu trách nhiệm định nghĩa route, kiểm tra đầu vào, quản lý hàng đợi streaming và điều phối các công việc xử lý. Các phần tính toán nặng được tách riêng thành module lõi gồm nạp mô hình, phát hiện phương tiện, phát hiện biểu số, theo dõi đối tượng,

nhận dạng ký tự, chấm điểm chất lượng và tổng hợp kết quả.

Khi ứng dụng khởi động, các mô hình học sâu nặng (như YOLO, OCR) được nạp một lần duy nhất dưới dạng không trạng thái (stateless) để dùng chung, kết nối cơ sở dữ liệu được khởi tạo và tác vụ dọn dẹp các phiên chọn trích đoạn tạm thời được kích hoạt. Trong khi đó, các thành phần có trạng thái (stateful) như bộ theo dõi phương tiện (BoT-SORT) sẽ được khởi tạo độc lập cho từng phiên xử lý (per-session). Cách tổ chức này vừa giúp tiết kiệm bộ nhớ VRAM do không phải nhân bản các mô hình lớn, vừa đảm bảo dữ liệu theo dõi (bộ lọc Kalman, định danh track) của các video xử lý đồng thời không bị xung đột, đồng thời giải phóng tài nguyên an toàn khi phiên kết thúc.

| Nhóm API                   | Chức năng                                                                                                                                                                                                                                           |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Xác thực                   | Tạo tài khoản, đăng nhập, đăng xuất, kiểm tra phiên người dùng và cấp token bảo mật cho các yêu cầu ghi dữ liệu.                                                                                                                                    |
| Xử lý video tải lên        | Tiếp nhận video, tạo công việc xử lý bất đồng bộ, phát tiến độ và kết quả theo luồng thời gian thực, cung cấp khung hình preview và kết quả chi tiết của từng phương tiện.                                                                          |
| Lịch sử nhận dạng          | Liệt kê các phiên xử lý đã lưu, lấy thông tin tổng quan một phiên và danh sách kết quả nhận dạng; chỉ trả dữ liệu thuộc về người dùng hiện tại.                                                                                                     |
| Chọn trích đoạn và sự kiện | Tạo và quản lý phiên chọn trích đoạn từ file tải lên hoặc nguồn camera trực tiếp, tiếp nhận lệnh đánh dấu sự kiện, xử lý đoạn video tương ứng ở backend, phát tiến độ và kết quả theo luồng, đồng thời cung cấp API tra cứu lại các sự kiện đã lưu. |

**Bảng 10:** Các nhóm API chính của backend

Luồng xử lý video được thiết kế theo cơ chế bất đồng bộ. Backend kiểm tra định dạng, dung lượng và tính hợp lệ của file đầu vào, sau đó sinh mã định danh công việc, tạo hàng đợi kết quả và khởi chạy pipeline trong một luồng riêng. Để đảm bảo tính ổn định trên máy chủ GPU, hệ thống sử dụng cơ chế kiểm soát đồng thời (bằng Semaphore) nhằm giới hạn số lượng tác vụ nhận dạng chạy song song (ví dụ: tối đa 2 video cùng lúc để tránh lỗi tràn bộ nhớ VRAM), và trả về mã lỗi HTTP 429 nếu vượt quá ngưỡng tải. Trong quá trình xử lý, backend đẩy liên tục các bản tin gồm tiến độ theo frame, khung hình preview, kết quả từng phương tiện và trạng thái hoàn thành theo chuẩn SSE, nhờ đó giao diện có thể cập nhật gần thời gian thực mà không cần hỏi vòng lặp.

Về bảo mật, mật khẩu người dùng được băm bằng thuật toán bcrypt, phiên đăng nhập được lưu phía máy chủ và token xác thực được gửi về trình duyệt trong cookie bảo mật chỉ đọc qua HTTP. Các yêu cầu ghi dữ liệu trong luồng xử lý video chính yêu cầu thêm token chống giả mạo yêu cầu trong header. Cách triển khai này tách dữ liệu xác thực khỏi mã nguồn giao diện và tránh lưu token truy cập trong bộ nhớ trình duyệt.

Lớp lưu trữ được tách khỏi lớp xử lý nghiệp vụ. Cơ sở dữ liệu lưu thông tin người dùng, phiên xử lý, kết quả nhận dạng và sự kiện được đánh dấu. Object storage lưu ảnh minh chứng để tránh lưu dữ liệu ảnh trực tiếp trong document. Nhờ cách tách này, backend API giữ vai trò điều phối, còn dữ liệu lớn và metadata được lưu ở các thành phần phù hợp.

#### **0.4.4 Thiết kế cơ sở dữ liệu**

Hệ thống sử dụng MongoDB làm cơ sở dữ liệu chính và một dịch vụ object storage tương thích S3 làm kho lưu trữ ảnh minh chứng. MongoDB lưu toàn bộ dữ liệu có cấu trúc gồm tài khoản người dùng, phiên đăng nhập, phiên xử lý video, kết quả nhận dạng và các sự kiện được đánh dấu trong chế độ chọn trích đoạn. Ảnh crop phương tiện, ảnh crop biển số và các khung hình minh chứng được lưu trên object storage; trong MongoDB chỉ lưu URL trỏ tới các đối tượng này.

Thiết kế này bám theo cách dữ liệu được sinh ra trong pipeline: một phiên xử lý có thể sinh nhiều phương tiện, mỗi phương tiện có một biển số cuối cùng và nhiều crop theo thời gian. Nếu lưu ảnh nhị phân trực tiếp trong document, kích thước tăng nhanh và khó mở rộng. Vì vậy hệ thống tách metadata khỏi ảnh: cơ sở dữ liệu giữ khóa truy vấn, kết quả OCR và các URL; object storage giữ dữ liệu ảnh.

**Các collection chính** Bảng ?? mô tả ánh xạ giữa các thực thể nghiệp vụ và collection trong MongoDB.

**Bảng 11:** Các collection và document trong MongoDB

| Thực thể             | Collection           | Mô tả                                                                                                                                                                                                |
|----------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tài khoản người dùng | users                | Lưu thông tin đăng ký, email duy nhất, mật khẩu đã băm và trạng thái tài khoản.                                                                                                                      |
| Phiên đăng nhập      | auth_sessions        | Lưu phiên đăng nhập phía máy chủ; tham chiếu đến tài khoản người dùng và hỗ trợ thu hồi phiên.                                                                                                       |
| Phiên nhận dạng      | recognition_sessions | Lưu metadata của một lượt xử lý video: tên file nguồn, trạng thái, số frame và chế độ xử lý.                                                                                                         |
| Kết quả nhận dạng    | recognition_records  | Lưu kết quả cuối cùng của từng phương tiện: biển số tổng hợp, độ tin cậy, lớp xe và URL ảnh minh chứng. Mỗi record nhúng thêm danh sách crop biển số theo thời gian phục vụ màn hình xem minh chứng. |
| Sự kiện video        | events               | Lưu một cửa sổ thời gian được người dùng đánh dấu trên nguồn video, gồm loại nguồn, khoảng thời gian, trạng thái xử lý và danh sách phương tiện phát hiện được nhúng trực tiếp trong document.       |
| Ảnh minh chứng       | (Object Storage)     | Không phải collection MongoDB. File ảnh lưu trên object storage; MongoDB chỉ lưu URL tham chiếu trong các trường tương ứng của kết quả nhận dạng và sự kiện.                                         |

**Thiết kế chỉ mục** Bảng ?? liệt kê các chỉ mục chính được tạo để phục vụ các truy vấn thường gặp của ứng dụng.



**Bảng 12:** Các chỉ mục MongoDB chính

| Collection           | Chỉ mục                                                                      | Mục đích                                                                  |
|----------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| users                | email (duy nhất);<br>created_at giảm dần                                     | Đảm bảo email không trùng và tăng tốc đăng nhập.                          |
| auth_sessions        | session_id (duy nhất);<br>user_id; expires_at                                | Kiểm tra và truy vết phiên đăng nhập theo người dùng.                     |
| recognition_sessions | session_id (duy nhất);<br>(user_id, created_at);<br>status                   | Lấy phiên theo mã, liệt kê lịch sử gần nhất và lọc theo trạng thái.       |
| recognition_records  | session_id; (session_id,<br>track_id) duy nhất;<br>plate_text; vehicle_class | Lấy kết quả theo phiên, tránh lưu trùng track và hỗ trợ tìm kiếm biển số. |
| events               | event_id (duy nhất);<br>session_id; marked_at<br>giảm dần; status            | Lấy chi tiết sự kiện, liệt kê theo phiên và theo thời gian đánh dấu.      |

## 0.5 Triển khai

Ứng dụng được triển khai theo mô hình tách riêng frontend, backend xử lý AI và các dịch vụ lưu trữ. Frontend được triển khai trên Vercel, backend được đóng gói bằng Docker và chạy trên RunPod GPU. Cơ sở dữ liệu và lưu trữ ảnh kết quả sử dụng các dịch vụ bên ngoài để giảm tải cho server xử lý chính.

| Thành phần                 | Cấu hình triển khai                |
|----------------------------|------------------------------------|
| Frontend                   | React/Vite, triển khai trên Vercel |
| Backend API                | FastAPI, chạy trên cổng 8000       |
| Server xử lý AI            | RunPod GPU, NVIDIA RTX 3090        |
| Môi trường GPU             | PyTorch 2.4.1, CUDA 12.1, cuDNN 9  |
| Streaming video            | MediaMTX, sử dụng WebRTC/RTSP      |
| Cơ sở dữ liệu              | MongoDB Atlas                      |
| Lưu trữ ảnh kết quả        | Supabase Storage                   |
| Giới hạn dung lượng upload | 512 MB                             |

**Bảng 13:** Cấu hình triển khai ứng dụng

Trong môi trường thử nghiệm cục bộ, hệ thống được chạy bằng Docker Compose gồm bốn dịch vụ chính: backend API, frontend web, MediaMTX và MongoDB. Backend mở cổng 8000, frontend mở cổng 3000, MongoDB mở cổng 27017, MediaMTX sử dụng các cổng 8554, 8889, 8189/udp và 9997. Cấu hình này phục vụ kiểm thử chức năng đăng nhập, tải video, xử lý nhận dạng và xem kết quả

trực tiếp.

Trong môi trường triển khai thử nghiệm trên server GPU, backend và MediaMTX được đóng gói trong cùng một Docker image. Backend đảm nhiệm các bước phát hiện phương tiện, phát hiện biển số, tracking, OCR và trả kết quả qua API. MediaMTX phục vụ luồng video trực tiếp, hỗ trợ hiển thị nguồn camera trong chế độ chọn trích đoạn trên giao diện web. Các mô hình như detector phương tiện, detector biển số, OCR và ReID được đặt trên network volume của RunPod.